Competitive Programming Maths

Contents

1 Number Bases and Binary Mathematics **4**
- 1.1 What Are Number Bases? . . . 4
- 1.2 Binary Numbers . . . 4
- 1.3 Bitwise Operators . . . 4
- 1.4 Binary Tricks . . . 4
- 1.5 C++ Built-in Functions . . . 6
- 1.6 What is $\log_2$? . . . 6
- 1.7 Edge Cases . . . 6

2 Data Types and Precision in C++ **7**
- 2.1 Why Data Types Matter . . . 7
- 2.2 Integer Data Types . . . 7
- 2.3 Floating Point Data Types . . . 7
- 2.4 Floating Point Inaccuracies (With Examples) . . . 8
- 2.5 Comparing Floating Point Numbers . . . 8
- 2.6 Competitive Programming Notes . . . 9
- 2.7 Why `int` can give Runtime Error . . . 9

3 Modular Arithmetic and Binary Exponentiation **10**
- 3.1 What is Modular Arithmetic? . . . 10
- 3.2 Basic Properties of Modulo . . . 10
- 3.3 Binary Exponentiation . . . 10
- 3.4 Modular Inverse . . . 11
- 3.5 Modular Inverse Using Fermat's Little Theorem . . . 11
- 3.6 Reference Implementation . . . 11
- 3.7 Common Pitfalls . . . 12

4 Greatest Common Divisor (GCD) and Euclidean Algorithm **12**
- 4.1 What is GCD? . . . 12
- 4.2 Properties of GCD . . . 13
- 4.3 Euclidean Algorithm . . . 13
- 4.4 Proof of Euclidean Algorithm . . . 13
- 4.5 Time Complexity . . . 13
- 4.6 C++ Implementation . . . 14
- 4.7 Practice Problem: Yet Another Array Problem . . . 14

5 Prime Numbers and Factorization Techniques **16**
5.1 Prime Numbers . . . 16
5.2 Primality Testing . . . 16
5.3 Generating All Prime Numbers up to a Limit . . . 17
5.4 Prime Factorization of a Single Number . . . 18
5.5 Finding All Divisors of a Number . . . 18
5.6 Finding All Divisors of All Numbers up to N . . . 19
5.7 Prime Factors of All Numbers up to N . . . 19

6 Smallest Prime Factor (SPF) **20**
6.1 What is the Smallest Prime Factor? . . . 20
6.2 Why Do We Need SPF? . . . 20
6.3 Core Idea Behind SPF . . . 20
6.4 Why Starting from i^2 Works . . . 21
6.5 What Does SPF Give Us? . . . 21
6.6 Time and Space Complexity . . . 21
6.7 SPF Construction . . . 21
6.8 Applications of SPF . . . 22
6.9 Practice Problem: Reducing Fractions . . . 22

7 Combinatorics **24**
7.1 Factorials . . . 24
7.2 Precomputation of Factorials . . . 24
7.3 Precomputation of Inverse Factorials . . . 25
7.4 Practice Problem: Christmas Tree Decoration . . . 26
7.5 Computing nCr and nPr Using Precomputed Arrays . . . 27
7.6 Computing nCr Without Precomputation . . . 27
7.7 Practice Problem: Unfair Game . . . 28
7.8 Practice Problem: Production of Snowmen . . . 29
7.9 Binomial Theorem . . . 30
7.10 Stars and Bars . . . 30
7.11 Practice Problem: Stars & Bars . . . 31

8 Probability and Expectation **31**
8.1 Probability Basics . . . 31
8.2 Probability Under Modulo . . . 32
8.3 Conditional Probability . . . 32
8.4 Independent Events . . . 32
8.5 Expected Value . . . 32
8.6 Linearity of Expectation . . . 32
8.7 Probability DP . . . 32
8.8 Practice Problem (Probability) . . . 32

9 Series and Summations **34**
9.1 Arithmetic Progression (AP) . . . 34
9.2 Practice Problem (Arithmetic Progression) . . . 34
9.3 Geometric Progression (GP) . . . 34
9.4 Practice Problem (Geometric Progression) . . . 35
9.5 Common Summation Formulas . . . 35
9.6 Telescoping Series . . . 36
9.7 Prefix Sums . . . 36
9.8 Prefix XOR . . . 37

9.9 Suffix Sums and XOR . . . 38

10 Pigeonhole Principle **38**
10.1 Statement . . . 38
10.2 Why This Works (Intuition) . . . 38
10.3 Simple Examples . . . 39
10.4 Classic CP Use Case . . . 39
10.5 Practice Problem (Pigeonhole Principle) . . . 39

11 Basic Geometry for Competitive Programming **40**
11.1 Distance Between Two Points . . . 40
11.2 Practice Problem (Geometry) . . . 41

12 Basic Statistics **41**
12.1 Mean . . . 41
12.2 Median . . . 42
12.3 Practice Problem (Median) . . . 42
12.4 Practice Problem (Median & Manhattan Distance) . . . 43
12.5 Mode . . . 44
12.6 Practice Problem: Majority Element . . . 44
12.7 Practice Problem: Majority Element II . . . 44

13 Additional Practice Problems **45**

1 Number Bases and Binary Mathematics

1.1 What Are Number Bases?

1.1.1 Definition

A **number base** (or radix) is the number of distinct digits used to represent numbers, including zero. In base b, each digit position represents a power of b.

$$(d_k d_{k-1} \dots d_1 d_0)_b = \sum_{i=0}^{k} d_i \cdot b^i$$

1.2 Binary Numbers

1.2.1 Representation

Binary numbers use base 2.

$$(101101)_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 45$$

1.2.2 Key Properties

- Each bit represents a power of 2
- k bits represent values from 0 to $2^k - 1$
- LSB decides parity

1.3 Bitwise Operators

Operator	Symbol	Meaning
AND	`&`	Both bits must be 1
OR	`|`	Any bit is 1
XOR	`^`	Bits differ
NOT	`~`	Flip bits
Left Shift	`<<`	Multiply by 2^k
Right Shift	`>>`	Divide by 2^k

1.4 Binary Tricks

1.4.1 Power of Two Check

$$n > 0 \wedge (n \& (n-1)) = 0$$

Why it works: A power of two has exactly one set bit. Subtracting 1 flips that bit and all bits after it, so AND becomes zero.

—

1.4.2 Lowest Set Bit

$$n\&(-n)$$

Meaning: Extracts the value of the least significant set bit.
Example:

$$n = 12 = (1100)_2 \Rightarrow n\&(-n) = 4$$

—

1.4.3 Important XOR Identities

- $a \oplus a = 0$
- $a \oplus 0 = a$
- $a \oplus b \oplus a = b$
- XOR is **commutative** and **associative**

—

1.4.4 AND Operator Properties

- $a\&a = a$
- $a\&0 = 0$
- $a\&b \leq \min(a, b)$

Observation: AND only keeps bits that are set in **both** numbers.
—

1.4.5 OR Operator Properties

- $a|a = a$
- $a|0 = a$
- $a|b \geq \max(a, b)$

Observation: OR sets a bit if it appears in **either** number.
—

1.4.6 Bitwise NOT

$$\sim a = -a - 1$$

Why this holds: Integers are stored in two's complement.
Example:

$$a = 5 \Rightarrow \sim a = -6$$

—

1.4.7 Useful CP Identities

- $(a \oplus b) \oplus c = a \oplus (b \oplus c)$
- $a + b = (a \oplus b) + 2 \cdot (a\&b)$
- Removing lowest set bit:

$$n = n\&(n-1)$$

—

1.5 C++ Built-in Functions

1.5.1 Popcount

```
int x = 13;
int bits = __builtin_popcount(x); // 3
```

1.5.2 Leading / Trailing Zeros

```
int x = 16;
__builtin_ctz(x); // 4
__builtin_clz(x); // depends on bit-width
```

1.6 What is $\log_2$?

1.6.1 Definition

$$\log_2(n) = \text{the power to which 2 must be raised to obtain } n$$

1.6.2 CP Interpretation

- Number of bits required to represent n
- Number of times we can divide n by 2 before it becomes zero

1.6.3 Properties of $\log_2$

For positive numbers a, b:

- $\log_2(1) = 0$
- $\log_2(2^k) = k$
- $\log_2(ab) = \log_2(a) + \log_2(b)$
- $\log_2\left(\frac{a}{b}\right) = \log_2(a) - \log_2(b)$
- $\log_2(a^k) = k \log_2(a)$

1.6.4 Important Inequality

$$2^{\lfloor \log_2(n) \rfloor} \leq n < 2^{\lfloor \log_2(n) \rfloor + 1}$$

1.6.5 Relation with Binary Length

The number of bits required to represent a positive integer n is:

$$\lfloor \log_2(n) \rfloor + 1$$

1.7 Edge Cases

- `__builtin_clz(0)` is undefined
- Shifting negative integers is implementation-defined
- Avoid floating-point `log2` due to precision issues
- Prefer bitwise operations for exact results

2 Data Types and Precision in C++

2.1 Why Data Types Matter

Choosing the correct data type is critical in competitive programming because:

- Overflow bugs silently produce wrong answers
- Bitwise operations depend on exact binary representation
- Floating-point precision errors can break geometry and math problems

2.2 Integer Data Types

2.2.1 Signed Integers

Type	Bits	Decimal Range	Binary Representation
`int`	32	$[-2^{31}, 2^{31}-1]$	Two's Complement
`long long`	64	$[-2^{63}, 2^{63}-1]$	Two's Complement

Two's Complement Representation Signed integers are stored using **two's complement**:

- MSB indicates sign
- Negative numbers are represented as:

$$-x = \sim x + 1$$

2.2.2 Unsigned Integers

Type	Bits	Decimal Range	Binary Range
`unsigned int`	32	$[0, 2^{32}-1]$	$[0, 111\ldots111_2]$
`unsigned long long`	64	$[0, 2^{64}-1]$	$[0, 111\ldots111_2]$

CP Rule Use `long long` by default.

2.3 Floating Point Data Types

2.3.1 Types and Precision

Type	Size	Significant Digits	Exponent Range
`float`	32-bit	~ 6	$\sim 10^{\pm 38}$
`double`	64-bit	~ 15	$\sim 10^{\pm 308}$
`long double`	80-bit (gcc)	~ 18	Larger

2.3.2 IEEE 754 Representation (Conceptual)

A floating-point number is stored as:

$$(-1)^{sign} \times mantissa \times 2^{exponent}$$

Because mantissa has limited bits, many decimals cannot be represented exactly.

2.4 Floating Point Inaccuracies (With Examples)

2.4.1 Why Floating Point Inaccuracies Occur

Many decimal numbers cannot be represented exactly in binary.

$$0.1_{10} = 0.00011001100110011\ldots_2$$

Since the binary expansion is infinite, it must be truncated, leading to approximation errors.

2.4.2 Example 1: Unexpected Output

```
double x = 0.1 + 0.2;
cout << fixed << setprecision(17) << x;
```

Output:

$$0.30000000000000004$$

2.4.3 Example 2: Equality Comparison Failure

```
double a = 0.3;
double b = 0.1 + 0.2;

if (a == b)
    cout << "Equal";
else
    cout << "Not Equal";
```

Output:

`Not Equal`

2.5 Comparing Floating Point Numbers

2.5.1 Why == is Dangerous

- Floating-point numbers are approximations
- Exact equality rarely holds

2.5.2 Epsilon-Based Comparison

Definition

$$\varepsilon = \text{a very small positive number}$$

Rule

$$|a - b| < \varepsilon \Rightarrow a \approx b$$

Typical Values

- `1e-9` for `double`
- `1e-12` or smaller for `long double`

2.5.3 C++ Implementation

```
const double EPS = 1e-9;

bool equal(double a, double b) {
    return fabs(a - b) < EPS;
}
```

2.6 Competitive Programming Notes

- Use `long long` everywhere by default to avoid overflow
- `int` operations are faster than `long long`, but safety matters more
- Prefer integer arithmetic over floating-point whenever possible
- Comparing floating-point numbers using `==` leads to incorrect results
- `__builtin_clz(0)` and similar functions are undefined
- Shifting negative integers is implementation-defined

2.7 Why `int` can give Runtime Error

2.7.1 Problem Reference (CP Context)

This issue appears in the following real competitive programming problem:

- **Divide Two Integers**
- Platform: LeetCode
- Link: `https://leetcode.com/problems/divide-two-integers/description/`

2.7.2 The Problematic Case

Consider the division:

$$-2^{31} \div (-1)$$

Mathematically:

$$-2^{31} \div (-1) = 2^{31}$$

However,

$$2^{31} = 2147483648 > \texttt{INT_MAX}$$

2.7.3 Why This Causes a Runtime Error

A 32-bit signed integer (`int`) has the range:

$$[-2^{31},\ 2^{31} - 1]$$

The value 2147483648 cannot be represented using `int`. In C++, this results in **undefined behavior**.

2.7.4 Failing Code Example

```cpp
int divide(int dividend, int divisor) {
    return dividend / divisor;
}
```

Input:

`dividend = -2147483648, divisor = -1`

Observed Result:

- Runtime Error
- Integer overflow
- Undefined behavior

2.7.5 Correct Approach Using `long long`

```
int divide(int dividend, int divisor) {
    return min(2147483647LL,(long long)dividend / divisor));
}
```

3 Modular Arithmetic and Binary Exponentiation

3.1 What is Modular Arithmetic?

3.1.1 Definition

For a positive integer m, we say:

$$a \equiv b \pmod{m}$$

if $(a - b)$ is divisible by m.

In competitive programming, answers are often required modulo a fixed number, most commonly:

$$m = 10^9 + 7 \quad \text{or} \quad m = 998244353$$

3.1.2 Why Modulo is Used

- Prevents integer overflow
- Keeps values within fixed bounds
- Required explicitly in many problems

3.2 Basic Properties of Modulo

For any integers a, b:

$$\begin{aligned}
(a + b) \bmod m &= ((a \bmod m) + (b \bmod m)) \bmod m \\
(a - b) \bmod m &= ((a \bmod m) - (b \bmod m) + m) \bmod m \\
(a \cdot b) \bmod m &= ((a \bmod m) \cdot (b \bmod m)) \bmod m
\end{aligned}$$

3.2.1 Important Warning

$$(a/b) \bmod m \neq (a \bmod m)/(b \bmod m)$$

Division under modulo requires a **modular inverse**.

3.3 Binary Exponentiation

3.3.1 Problem

Compute:

$$a^b \bmod m$$

efficiently.

A naive approach takes $O(b)$ time, which is too slow.

3.3.2 Idea

Represent the exponent in binary and repeatedly square the base.

3.3.3 Algorithm

```
long long binpow(long long a, long long b, long long m) {
    long long res = 1;
    a %= m;
    while (b) {
        if (b & 1) res = (res * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return res;
}
```

3.3.4 Time Complexity

$$O(\log b)$$

3.4 Modular Inverse

3.4.1 Definition

The modular inverse of a modulo m is a number x such that:

$$a \cdot x \equiv 1 \pmod{m}$$

3.4.2 Existence Condition

A modular inverse exists if and only if:

$$\gcd(a, m) = 1$$

3.5 Modular Inverse Using Fermat's Little Theorem

3.5.1 Theorem

If m is prime and $\gcd(a, m) = 1$, then:

$$a^{m-1} \equiv 1 \pmod{m}$$

3.5.2 Proof

From Fermat's Little Theorem:

$$a^{m-1} \equiv 1 \pmod{m}$$

Multiplying both sides by a^{-1}:

$$a^{m-2} \equiv a^{-1} \pmod{m}$$

Hence,

$$\boxed{a^{-1} \equiv a^{m-2} \pmod{m}}$$

3.6 Reference Implementation

3.6.1 Modular Power

```
long long modpow(long long a, long long b, long long m) {
    long long res = 1;
    a %= m;
    while (b) {
        if (b & 1) res = (res * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return res;
}
```

3.6.2 Modular Inverse

```
long long inv(long long a, long long m) {
    return modpow(a, m - 2, m);
}
```

3.6.3 Division Under Modulo

$$\frac{a}{b} \bmod m = a \cdot b^{-1} \bmod m$$

```
long long divide(long long a, long long b, long long m) {
    return (a % m) * inv(b, m) % m;
}
```

3.7 Common Pitfalls

- Modular inverse does not exist if $\gcd(a, m) \neq 1$
- Fermat's method works only for prime moduli
- Forgetting modulo during multiplication causes overflow
- Direct division under modulo is incorrect

4 Greatest Common Divisor (GCD) and Euclidean Algorithm

4.1 What is GCD?

4.1.1 Definition

The **Greatest Common Divisor (GCD)** of two integers a and b, denoted as $\gcd(a, b)$, is the largest positive integer that divides both a and b.

$$\gcd(a, b) = \max\{d \mid d \mid a \text{ and } d \mid b\}$$

4.1.2 Examples

$$\gcd(12, 18) = 6$$

$$\gcd(7, 13) = 1$$

If $\gcd(a, b) = 1$, the numbers are called **coprime**.

4.2 Properties of GCD

- $\gcd(a, 0) = |a|$
- $\gcd(0, 0)$ is undefined
- $\gcd(a, b) = \gcd(b, a)$
- $\gcd(a, b) = \gcd(b, a \bmod b)$
- $\gcd(a, b) \mid a$ and $\gcd(a, b) \mid b$
- If $d \mid a$ and $d \mid b$, then $d \mid \gcd(a, b)$

4.2.1 Relation with LCM

$$\gcd(a, b) \cdot \operatorname{lcm}(a, b) = |a \cdot b|$$

4.3 Euclidean Algorithm

4.3.1 Idea

The Euclidean Algorithm is based on the observation:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

We repeatedly replace the larger number by the remainder until it becomes zero.

4.3.2 Algorithm Steps

1. If $b = 0$, return a
2. Otherwise, compute $\gcd(b, a \bmod b)$

4.4 Proof of Euclidean Algorithm

Let $a = bq + r$, where $r = a \bmod b$.

- Any number dividing both a and b also divides $r = a - bq$
- Any number dividing both b and r also divides a

Hence, the set of common divisors of (a, b) and (b, r) is the same.

$$\Rightarrow \gcd(a, b) = \gcd(b, a \bmod b)$$

This justifies the correctness of the algorithm.

4.5 Time Complexity

Each step reduces the second argument.

$$\text{Time Complexity} = O(\log \min(a, b))$$

This makes the Euclidean Algorithm extremely efficient even for large numbers.

4.6 C++ Implementation

4.6.1 Recursive Version

```cpp
long long gcd(long long a, long long b) {
    if (b == 0)
        return a;
    return gcd(b, a % b);
}
```

4.6.2 Iterative Version

```cpp
long long gcd(long long a, long long b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}
```

4.6.3 Using C++ Built-in Function

```cpp
long long g = __gcd(a, b); // or std::gcd(a, b) in C++17+
```

4.7 Practice Problem: Yet Another Array Problem

4.7.1 Problem Reference

- **Problem**: D. Yet Another Array Problem
- Platform: Codeforces
- Link: `https://codeforces.com/contest/2167/problem/D`

4.7.2 Problem Summary

You are given an array a of length n. Find the smallest integer x ($2 \le x \le 10^{18}$) such that:

$$\exists i \text{ with } \gcd(a_i, x) = 1$$

If no such x exists, output -1.

—

4.7.3 Key Observation

If $\gcd(a_i, x) = 1$, then:

- x shares **no common prime factor** with a_i

So instead of reasoning about x directly, we reason about its **prime divisors**.

—

4.7.4 Important Claim

Let x be a valid answer and let p be the smallest prime divisor of x.

Then:

$$\gcd(p, a_i) = 1 \quad \text{for some } i$$

4.7.5 Proof of the Claim

Let:

$$x = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$$

and:

$$a_i = q_1^{f_1} \cdot q_2^{f_2} \cdots$$

We know:

$$\gcd(x, a_i) = \prod p_j^{\min(e_j, f_j)} = 1$$

Hence, for every prime divisor p_j of x, we must have:

$$f_j = 0$$

In particular, for the smallest prime divisor p of x:

$$\gcd(p, a_i) = 1$$

Thus, it is sufficient to check only **prime numbers** as candidates for x.

—

4.7.6 Why Brute Force Over Small Primes Works

We are asked for the **smallest** such x.

Observe:

$$2 \cdot 3 \cdot 5 \cdot 7 \cdots 53 > 10^{18}$$

So any number $x \leq 10^{18}$ must have a prime factor ≤ 53.

Therefore:

- It is sufficient to check primes up to 53
- If none work, answer is -1

—

4.7.7 Algorithm

1. Generate all primes ≤ 53
2. For each prime p:
   - Check if $\exists a_i$ such that $\gcd(a_i, p) = 1$
   - If yes, output p
3. If no prime satisfies the condition, output -1

—

4.7.8 Time Complexity

$$O(\text{number of primes} \times n) \approx O(16 \cdot n)$$

This easily fits within constraints.

—

5 Prime Numbers and Factorization Techniques

5.1 Prime Numbers

5.1.1 Definition

A **prime number** is an integer greater than 1 that has exactly two positive divisors:

$$1 \text{ and itself}$$

Numbers greater than 1 that are not prime are called **composite**.

5.2 Primality Testing

5.2.1 Brute Force Primality Test

Idea Check whether n is divisible by every integer from 2 to $n-1$.

Why It Works If n has a divisor other than 1 and itself, it must appear in this range.

Time Complexity

$$O(n)$$

Code

```cpp
bool isPrime(long long n) {
    if (n <= 1) return false;
    for (long long i = 2; i < n; i++)
        if (n % i == 0)
            return false;
    return true;
}
```

—

5.2.2 Optimization 1: Checking up to $\sqrt{n}$

Key Observation If $n = a \cdot b$, then at least one of $a, b \leq \sqrt{n}$.

What This Improves Reduces checks from n to $\sqrt{n}$.

Time Complexity

$$O(\sqrt{n})$$

Code

```cpp
bool isPrime(long long n) {
    if (n <= 1) return false;
    for (long long i = 2; i * i <= n; i++)
        if (n % i == 0)
            return false;
    return true;
}
```

—

5.2.3 Optimization 2: $6k \pm 1$ Method

Observation All integers can be written as:

$$6k,\ 6k \pm 1,\ 6k \pm 2,\ 6k + 3$$

Numbers divisible by 2 or 3 are composite. Thus, all primes greater than 3 are of the form $6k \pm 1$.

Time Complexity

$$O(\sqrt{n}/6)$$

Code

```
bool isPrime(long long n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    for (long long i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0)
            return false;

    return true;
}
```

5.3 Generating All Prime Numbers up to a Limit

5.3.1 Brute Force Method

Idea Check primality of every number from 2 to `limit`.

Why It Is Slow Each primality check costs $O(\sqrt{n})$.

Time Complexity

$$O(n\sqrt{n})$$

—

5.3.2 Optimized Method: Sieve of Eratosthenes

Core Idea If i is prime, all multiples of i are composite.

Why Start from i^2 Smaller multiples already have smaller prime factors.

What It Gives All prime numbers $\leq$ `limit`.

Time Complexity

$$O(n \log \log n)$$

Code

```
vector<long long> generatePrimesUpTo(long long limit) {
    vector<bool> isPrime(limit + 1, true);
    vector<long long> primes;

    isPrime[0] = isPrime[1] = false;
    for (long long i = 2; i * i <= limit; i++) {
        if (isPrime[i]) {
            for (long long j = i * i; j <= limit; j += i)
                isPrime[j] = false;
        }
    }

    for (long long i = 2; i <= limit; i++)
        if (isPrime[i]) primes.push_back(i);

    return primes;
}
```

5.4 Prime Factorization of a Single Number

5.4.1 Brute Force Factorization

Idea Try dividing n by all numbers from 2 to n.

Time Complexity

$$O(n)$$

—

5.4.2 Optimized Prime Factorization

Key Observation After removing all factors $\leq \sqrt{n}$, at most one prime factor remains.

Time Complexity

$$O(\sqrt{n})$$

Code

```
vector<long long> primeFactorize(long long n) {
    vector<long long> factors;
    for (long long i = 2; i * i <= n; i++) {
        while (n % i == 0) {
            factors.push_back(i);
            n /= i;
        }
    }
    if (n > 1)
        factors.push_back(n);
    return factors;
}
```

5.5 Finding All Divisors of a Number

5.5.1 Brute Force Method

Idea Check all numbers from 1 to n.

Time Complexity

$$O(n)$$

—

5.5.2 Optimized Method

Key Observation Divisors come in pairs:

$$d \cdot \frac{n}{d} = n$$

Time Complexity

$$O(\sqrt{n})$$

Code

```
vector<long long> getAllDivisors(long long n) {
    vector<long long> divs;
    for (long long i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            divs.push_back(i);
            if (i != n / i)
                divs.push_back(n / i);
        }
    }
    sort(divs.begin(), divs.end());
    return divs;
}
```

5.6 Finding All Divisors of All Numbers up to N

5.6.1 Brute Force Idea

Time Complexity

$$O(N^2)$$

—

5.6.2 Optimized Divisor Sieve

Key Idea If i divides j, then i is a divisor of j.

Time Complexity

$$O(N \log N)$$

Code

```
vector<vector<long long>> allDivisorsUpToN(long long n) {
    vector<vector<long long>> divs(n + 1);
    for (long long i = 1; i <= n; i++)
        for (long long j = i; j <= n; j += i)
            divs[j].push_back(i);
    return divs;
}
```

5.7 Prime Factors of All Numbers up to N

5.7.1 Idea

If p is prime, then p is a prime factor of all multiples of p.

What It Gives Prime factor list for every number.

Time Complexity

$$O(N \log \log N)$$

Code

```
vector<vector<long long>>
primeFactorsUpToN(long long n, const vector<long long>& primes) {
    vector<vector<long long>> factors(n + 1);
    for (auto p : primes)
        for (long long j = p; j <= n; j += p)
            factors[j].push_back(p);
    return factors;
}
```

6 Smallest Prime Factor (SPF)

6.1 What is the Smallest Prime Factor?

6.1.1 Definition

For every integer $x \geq 2$, the **Smallest Prime Factor (SPF)** of x, denoted as $\text{spf}[x]$, is the smallest prime number that divides x.

Examples:

$$\text{spf}[2] = 2, \quad \text{spf}[6] = 2, \quad \text{spf}[15] = 3, \quad \text{spf}[17] = 17$$

If $\text{spf}[x] = x$, then x is prime.

—

6.2 Why Do We Need SPF?

6.2.1 The Problem with Earlier Methods

- Trial division takes $O(\sqrt{n})$ per number
- Factoring many numbers becomes too slow
- Repeating work across queries is inefficient

SPF solves this by **preprocessing once** and answering queries fast.

—

6.3 Core Idea Behind SPF

6.3.1 Connection with the Sieve of Eratosthenes

SPF is an extension of the sieve.

- Initially, assume $\text{spf}[i] = i$
- When a prime i is found, mark it as the smallest divisor
- For each multiple $j = i^2, i^2 + i, \ldots$:

- If spf[j] is unset, set spf[j] $= i$

This ensures every number gets its smallest prime divisor.

—

6.4 Why Starting from i^2 Works

If $j < i^2$, then:

$$j = i \cdot k \quad \Rightarrow \quad k < i$$

So j would already have a smaller prime factor and its SPF would have been set earlier.

—

6.5 What Does SPF Give Us?

Using SPF, we can:

- Check if a number is prime in $O(1)$
- Factorize a number in $O(\log n)$
- Generate all primes up to N
- Precompute prime factors for all numbers

—

6.6 Time and Space Complexity

- Precomputation: $O(N \log \log N)$
- Space: $O(N)$
- Primality check: $O(1)$
- Factorization of one number: $O(\log n)$

—

6.7 SPF Construction

6.7.1 Code

```
vector<long long> computeSPF(long long n) {
    vector<long long> spf(n + 1);
    for (long long i = 1; i <= n; i++)
        spf[i] = i;

    for (long long i = 2; i * i <= n; i++) {
        if (spf[i] == i) { // i is prime
            for (long long j = i * i; j <= n; j += i) {
                if (spf[j] == j)
                    spf[j] = i;
            }
        }
    }
    return spf;
}
```

—

6.8 Applications of SPF

6.8.1 Primality Check

```
bool isPrime(long long x, const vector<long long>& spf) {
    return x >= 2 && spf[x] == x;
}
```

—

6.8.2 Generating All Primes up to N

```
vector<long long> generatePrimes(long long n,
                                 const vector<long long>& spf) {
    vector<long long> primes;
    for (long long i = 2; i <= n; i++) {
        if (spf[i] == i)
            primes.push_back(i);
    }
    return primes;
}
```

—

6.8.3 Prime Factorization Using SPF

```
vector<long long> primeFactors(long long n,
                               const vector<long long>& spf) {
    vector<long long> factors;
    while (n > 1) {
        factors.push_back(spf[n]);
        n /= spf[n];
    }
    return factors;
}
```

—

6.9 Practice Problem: Reducing Fractions

Problem Link `https://codeforces.com/contest/222/problem/C`

Problem Summary You are given two arrays representing the numerator and denominator of a fraction. Each array element contributes multiplicatively.

Your task is to **reduce the fraction** by cancelling common prime factors between the numerator and denominator, and output the reduced form.

The constraints allow values up to 10^7 and many numbers must be factorized.

—

6.9.1 Core Observation

Reduction of a fraction depends only on **prime factor counts**.

For each prime p:

- Let a_p = total exponent of p in the numerator
- Let b_p = total exponent of p in the denominator

After reduction:

$$\text{new } a_p = a_p - \min(a_p, b_p), \quad \text{new } b_p = b_p - \min(a_p, b_p)$$

So the problem becomes:

- Efficiently factorize many numbers
- Maintain prime exponent counts
- Reconstruct reduced numbers

—

6.9.2 Approach 1: Using Smallest Prime Factor (Recommended)

Idea

- Precompute the **Smallest Prime Factor (SPF)** for all numbers up to 10^7
- Factorize each number in $O(\log n)$ using SPF
- Accumulate prime exponent counts for numerator and denominator
- Cancel common prime powers
- Rebuild the reduced arrays

Why This Works Well

- SPF allows factorization by repeatedly dividing by the smallest prime
- Each division strictly reduces the number
- Total operations per number are proportional to the number of prime factors

Time Complexity

$$\text{Preprocessing: } O(N \log \log N)$$
$$\text{Factorization: } O(\log n) \text{ per number}$$

This approach is fast, scalable, and safe for tight constraints.

—

6.9.3 Approach 2: Without SPF (Trial Division)

Idea

- Generate all primes up to $\sqrt{10^7}$
- Factorize each number by trying division with all primes
- Accumulate and cancel prime exponents

Why It Is Slower

- Trial division may check many primes for each number
- Worst-case factorization takes $O(\sqrt{n})$

- Repeated work across many elements

Time Complexity

$$\text{Prime generation: } O(N \log \log N)$$
$$\text{Factorization: } O(\sqrt{n}) \text{ per number}$$

This approach can pass, but is close to time limits.

—

6.9.4 Comparison of Approaches

Aspect	SPF-Based	Trial Division
Accepted	Yes	Yes
Factorization Time	$O(\log n)$	$O(\sqrt{n})$
Speed	Fast	Slower
Memory Usage	Higher	Lower
Ease of Implementation	Moderate	Simple
Recommended	**Yes**	Only for small limits

—

7 Combinatorics

7.1 Factorials

7.1.1 Definition

$$n! = 1 \cdot 2 \cdot 3 \cdots n$$

—

7.2 Precomputation of Factorials

7.2.1 Why Precompute

Many problems require repeated computation of combinations and permutations. Precomputing factorials avoids recomputation and makes queries efficient.

7.2.2 Complexity

$$O(N) \text{ time}, \quad O(N) \text{ space}$$

7.2.3 Code

```
vector<long long> computeFactorials(int n, long long mod) {
    vector<long long> fact(n + 1);
    fact[0] = 1;
    for (int i = 1; i <= n; i++)
        fact[i] = fact[i - 1] * i % mod;
    return fact;
}
```

—

7.3 Precomputation of Inverse Factorials

7.3.1 Why Inverse Factorials Are Needed

To compute combinations using:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

we need inverses of factorial values.

A naive approach would compute:

$$(r!)^{-1},\ ((n-r)!)^{-1}$$

for every query.

—

7.3.2 Why Precomputing Inverse Factorials Is Better

Naive Approach

- Factorials are precomputed
- For every query, compute inverse of $r!$ and $(n-r)!$
- Each inverse costs $O(\log mod)$

Total Cost For Q queries:

$$O(Q \log mod)$$

—

Optimized Approach

- Compute inverse of $n!$ once
- Use the relation:

$$(i-1)!^{-1} = i!^{-1} \cdot i$$

- Build all inverse factorials in linear time

Total Cost

$$O(N + \log mod) \text{ preprocessing}$$

$$O(1) \text{ per query}$$

This is significantly faster when there are many queries.

—

7.3.3 Code

```
vector<long long> computeInverseFactorials(
    int n, long long mod, const vector<long long>& fact) {

    vector<long long> ifact(n + 1);
    ifact[n] = modpow(fact[n], mod - 2, mod);

    for (int i = n - 1; i >= 0; i--)
        ifact[i] = ifact[i + 1] * (i + 1) % mod;

    return ifact;
}
```

7.4 Practice Problem: Christmas Tree Decoration

Problem D. Christmas Tree Decoration

Link `https://codeforces.com/problemset/problem/2190/D`

Problem Summary There are n people and $(n+1)$ boxes of decorations. Box 0 is a shared box, while box i belongs to person i.

People act in a given permutation order: each person must take one decoration either from their own box or from box 0. This process repeats cyclically until all decorations are used.

A permutation is called **fair** if it is possible to finish the process without ever encountering a situation where a person needs to take a decoration but both box 0 and their own box are empty.

Your task is to count the number of fair permutations.

—

7.4.1 Key Observation

Each person i can safely act as long as:

- They still have decorations in their own box, or
- Box 0 has decorations left

This creates a global dependency on:

- The total number of decorations
- How many people are forced to rely on box 0

The order in which people appear matters, but the process can be analyzed by grouping people based on how many times they must draw from box 0.

—

7.4.2 Core Idea

- Think in terms of **how many times each person needs help from box** 0
- The process is valid if box 0 never runs out before all forced usages are satisfied
- The problem reduces to counting permutations where these constraints hold

This transforms the problem into a **counting permutations with restrictions**, which is a classic combinatorics setup.

—

7.4.3 Approach

1. Count how many decorations must come from box 0
2. Determine how many people are *forced* to rely on box 0
3. Count the number of permutations where these people appear in an order that never exhausts box 0
4. Use combinations and factorials to count valid arrangements

All counting is done modulo 998244353.

—

7.5 Computing nCr and nPr Using Precomputed Arrays

7.5.1 nCr

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

```
long long nCr(int n, int r,
              const vector<long long>& fact,
              const vector<long long>& ifact,
              long long mod) {
    if (r < 0 || r > n) return 0;
    return fact[n] * ifact[r] % mod * ifact[n - r] % mod;
}
```

7.5.2 Time Complexity

$$O(1)$$

—

7.5.3 nPr

$$P(n,r) = \frac{n!}{(n-r)!}$$

```
long long nPr(int n, int r,
              const vector<long long>& fact,
              const vector<long long>& ifact,
              long long mod) {
    if (r < 0 || r > n) return 0;
    return fact[n] * ifact[n - r] % mod;
}
```

—

7.6 Computing nCr Without Precomputation

7.6.1 Idea

Use the multiplicative form of combinations:

$$\binom{n}{r} = \frac{n(n-1)(n-2)\cdots(n-r+1)}{r!}$$

Always compute with:

$$r = \min(r, n-r)$$

—

7.6.2 Code

```
long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    r = min(r, n - r);

    long long res = 1;
    for (int i = 1; i <= r; i++) {
        res = res * (n - r + i) / i;
    }
    return res;
}
```

7.7 Practice Problem: Unfair Game

Problem D. Unfair Game

Link `https://codeforces.com/contest/2162/problem/D`

Problem Summary Bob chooses a number a from 1 to n, where $n = 2^d$. Alice can repeatedly:

- Divide the number by 2 if it is even, or
- Subtract 1

After each move, Alice learns the exponent of the largest power of 2 dividing the current number.

Alice wants to reach 0 within at most k moves. Your task is to count how many starting numbers Alice **cannot** win from within k moves, assuming optimal play.

—

7.7.1 Key Observation

For any number $a > 0$, there exists a unique value:

$$x = v_2(a)$$

such that:

$$a \equiv 0 \pmod{2^x}, \quad a \not\equiv 0 \pmod{2^{x+1}}$$

This is exactly the number of trailing zeros in the binary representation of a.

—

7.7.2 Core Idea

- Each division by 2 removes one trailing zero
- Subtracting 1 turns an odd number into an even one
- The total number of moves needed depends only on the binary structure of a

Thus, the game reduces to:

- Analyzing binary lengths
- Counting how many numbers have a given number of trailing zeros

—

7.7.3 Why Combinations Appear

Numbers from 1 to 2^d can be classified by:

- Total bit length
- Positions of 1 bits

Counting how many numbers fall into each category involves:

$$\binom{d}{k}$$

This allows us to count valid starting numbers using combinations, without needing factorial precomputation due to small effective ranges.

—

7.7.4 Approach

1. Let $d = \log_2(n)$
2. Iterate over possible binary lengths
3. For each length, count numbers requiring more than k moves
4. Use nCr to count configurations efficiently

All calculations are done using direct multiplicative nCr.

—

7.8 Practice Problem: Production of Snowmen

Problem C. Production of Snowmen

Link `https://codeforces.com/contest/2182/problem/C`

Problem Summary You are given three cyclic arrays:

$$A = (a_1, \ldots, a_n),\ B = (b_1, \ldots, b_n),\ C = (c_1, \ldots, c_n)$$

A snowman is **stable** if:

$$a < b < c$$

You must choose starting indices (i, j, k) such that for every shift $t = 0 \ldots n-1$:

$$a_{i+t} < b_{j+t} < c_{k+t}$$

The task is to count how many such triples (i, j, k) exist.

—

7.8.1 Key Observation

Because the conveyors are **cyclic**, once (i, j, k) is chosen:

- All comparisons are shift-invariant
- Only relative alignment matters, not absolute position

Thus, the problem reduces to checking consistency of inequalities across all n positions.

—

7.8.2 Reformulation

For a fixed offset alignment:

$$(a_x, b_y, c_z)$$

we need:

$$\forall t: \quad a_{x+t} < b_{y+t} \text{ and } b_{y+t} < c_{z+t}$$

This splits into two independent conditions:

- Valid alignments of A with B
- Valid alignments of B with C

—

7.8.3 Counting Strategy

- Precompute all valid cyclic shifts where $a_i < b_j$
- Precompute all valid cyclic shifts where $b_j < c_k$
- Combine compatible shifts using counting

Each valid configuration contributes multiplicatively.

—

7.9 Binomial Theorem

7.9.1 Statement

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} b^k$$

7.9.2 General Term

$$T_{k+1} = \binom{n}{k} a^{n-k} b^k$$

7.9.3 Finding Coefficients

- Coefficient of $a^{n-k}b^k$: $\binom{n}{k}$
- Coefficient of x^k in $(ax+b)^n$: $\binom{n}{k} a^k b^{n-k}$

—

7.10 Stars and Bars

7.10.1 Problem Statement

Count the number of non-negative integer solutions to:

$$x_1 + x_2 + \cdots + x_n = k$$

—

7.10.2 Intuition

Represent k identical objects as stars:

$$\star\star\star\cdots\star$$

To divide them into n groups, place $n-1$ bars between stars. Each distinct arrangement corresponds to a unique solution.

—

7.10.3 Formula

$$\boxed{\binom{n+k-1}{k}}$$

—

7.10.4 Why This Works

- Total positions: $k + n - 1$
- Choose k positions for stars
- Remaining positions are bars

—

7.11 Practice Problem: Stars & Bars

Problem **CF 1288C — Two Arrays**

Link `https://codeforces.com/problemset/problem/1288/C`

Core Idea The problem asks for the number of ways to build an array with fixed sum constraints on all prefixes and total sum.

Key combinatorial insight:

- You are effectively distributing a fixed total sum across positions
- Each prefix has a minimum required sum
- This leads to counting integer solutions with lower bounds

This is a typical **Stars and Bars with constraints** scenario, where we adjust variables to account for minimum sums, then apply standard counting.

Why It Fits Stars and Bars solves counts of non-negative integer solutions:

$$x_1 + x_2 + \cdots + x_n = k$$

But when there are **prefix minimum constraints**, you:

- shift variables to remove lower bounds
- transform into a standard root counting problem
- apply combination formulas accordingly

—

8 Probability and Expectation

8.1 Probability Basics

8.1.1 Definition

$$P(A) = \frac{\text{favorable outcomes}}{\text{total outcomes}}$$

In competitive programming:

$$P \equiv \text{favorable} \times inv(\text{total}) \pmod{mod}$$

—

8.2 Probability Under Modulo

Division is replaced by modular inverse:

$$\frac{a}{b} \equiv a \cdot b^{mod-2} \pmod{mod}$$

—

8.3 Conditional Probability

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Used when outcomes depend on previous events.

—

8.4 Independent Events

Events A and B are independent if:

$$P(A \cap B) = P(A) \cdot P(B)$$

—

8.5 Expected Value

8.5.1 Definition

$$\mathbb{E}[X] = \sum x \cdot P(X = x)$$

—

8.6 Linearity of Expectation

$$\mathbb{E}[X_1 + X_2 + \cdots + X_n] = \sum_{i=1}^{n} \mathbb{E}[X_i]$$

—

8.7 Probability DP

$$dp[i] = \sum dp[j] \cdot P(j \to i)$$

—

8.8 Practice Problem (Probability)

Problem **CF 2125D — Segments Covering**

Link `https://codeforces.com/problemset/problem/2125/D`

Goal Compute the probability that **each cell is covered by exactly one segment**, where each segment exists independently with probability $\frac{p_i}{q_i}$.

8.8.1 Key Probability Transformation

For each segment with probability

$$P_i = \frac{p_i}{q_i},$$

rewrite it as:

$$P_i = (1 - P_i) \cdot \frac{P_i}{1 - P_i}.$$

This allows the total probability to be expressed as:

$$\left(\prod_i (1 - P_i)\right) \cdot \left(\sum \text{valid configurations using } \frac{P_i}{1 - P_i}\right).$$

- The product term accounts for all segments initially being inactive
- The summation term activates exactly one valid segment per cell

8.8.2 DP Interpretation

- Segments are grouped by their right endpoint
- Each segment contributes a weight:

$$w_i = \frac{P_i}{1 - P_i}$$

- A suffix DP is computed from right to left
- DP ensures that:
  - exactly one segment covers each cell
  - overlapping segments are handled without double counting

The DP value at position i represents the total contribution of valid segment selections covering suffix $[i, m - 1]$.

8.8.3 Why Reverse (Suffix) DP Works

Processing from right to left ensures:

- each segment contributes only to the cells it covers
- overlapping probabilities are merged correctly
- independence between segments is preserved algebraically

This avoids explicit enumeration of cells or subsets.

8.8.4 Final Formula

The final probability is:

$$\text{Answer} = \left(\prod_i (1 - P_i)\right) \cdot \text{DP}[0] \pmod{998244353}.$$

8.8.5 Complexity

- Time Complexity: $O(n + m)$
- Space Complexity: $O(n + m)$

9 Series and Summations

9.1 Arithmetic Progression (AP)

9.1.1 Nth Term

$$a_n = a + (n-1)d$$

9.1.2 Sum of First n Terms

$$S_n = \frac{n}{2}(2a + (n-1)d)$$

9.1.3 Special Case

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

9.2 Practice Problem (Arithmetic Progression)

Problem **CF 1616C — Representative Edges**

Link `https://codeforces.com/contest/1616/problem/C`

Key Observation An array is an arithmetic progression if and only if there exist constants a and d such that:

$$a_i = a + i \cdot d$$

Equivalently, for any two indices $i < j$:

$$d = \frac{a_j - a_i}{j - i}$$

—

Approach

- Fix two indices (i, j) to uniquely determine the common difference d
- This fixes the entire arithmetic progression
- For all positions k, check whether:

$$a_k = a_i + (k - i) \cdot d$$

- Count how many elements already match this AP
- Minimum operations = total elements − maximum matches

—

9.3 Geometric Progression (GP)

9.3.1 Nth Term

$$a_n = ar^{n-1}$$

9.3.2 Sum ($r \neq 1$)

$$S_n = a \cdot \frac{r^n - 1}{r - 1}$$

9.3.3 Infinite GP ($|r| < 1$)

$$S = \frac{a}{1 - r}$$

9.4 Practice Problem (Geometric Progression)

Problem **CF 2182B — New Year Cake**

Link `https://codeforces.com/contest/2182/problem/B`

Key Observation The sizes of cake layers follow a geometric progression:

$$1,\ 2,\ 4,\ 8,\ \ldots$$

Each layer is twice the size of the layer above.

Additionally:

- Chocolate types must alternate
- The amount of chocolate required for a layer equals its size

—

Approach

- Consider two cases:
  - Top layer starts with white chocolate
  - Top layer starts with dark chocolate
- For a fixed number of layers k:
  - Sum the GP terms assigned to white layers
  - Sum the GP terms assigned to dark layers
- Check whether both sums fit within available chocolate
- Maximize k

—

9.5 Common Summation Formulas

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

—

9.6 Telescoping Series

9.6.1 Definition

A series where most terms cancel out.

9.6.2 Example

$$\sum_{k=1}^{n} \left(\frac{1}{k} - \frac{1}{k+1} \right) = 1 - \frac{1}{n+1}$$

—

9.6.3 General Form

If:

$$a_k = f(k) - f(k+1)$$

Then:

$$\sum_{k=1}^{n} a_k = f(1) - f(n+1)$$

—

9.7 Prefix Sums

9.7.1 Definition

Given an array:

$$a_1, a_2, \ldots, a_n$$

The **prefix sum array** is defined as:

$$p_i = a_1 + a_2 + \cdots + a_i$$

with:

$$p_0 = 0$$

—

9.7.2 Why Prefix Sums Work

Using prefix sums, any subarray sum can be computed in constant time:

$$\sum_{i=l}^{r} a_i = p_r - p_{l-1}$$

This converts repeated summation from:

$$O(n) \to O(1)$$

after a single preprocessing step.

—

9.7.3 Time and Space Complexity

- Preprocessing: $O(n)$
- Each range query: $O(1)$
- Extra space: $O(n)$

—

9.8 Prefix XOR

9.8.1 Definition

For bitwise XOR:

$$px_i = a_1 \oplus a_2 \oplus \cdots \oplus a_i$$

with:

$$px_0 = 0$$

—

9.8.2 Key Property

Using XOR:

$$a_l \oplus a_{l+1} \oplus \cdots \oplus a_r = px_r \oplus px_{l-1}$$

This works because:

$$x \oplus x = 0$$

—

9.8.3 Practice Problem (Prefix XOR Construction)

Problem **CF 2175B — XOR Array**

Link `https://codeforces.com/contest/2175/problem/B`

Problem Insight The XOR of a subarray can be expressed using prefix XOR:

$$a_l \oplus a_{l+1} \oplus \cdots \oplus a_r = px_r \oplus px_{l-1}$$

Thus, enforcing XOR conditions on subarrays translates into constraints on prefix XOR values.

—

Core Idea The problem requires:

- Exactly one subarray (l, r) whose XOR is 0
- All other subarrays must have non-zero XOR

Using prefix XOR:

$$px_r = px_{l-1}$$

must hold for exactly one pair of indices, and all other prefix XOR pairs must be distinct.

—

Construction Strategy

- Assign prefix XORs such that:
  - $px_{l-1} = px_r$
  - All other prefix XOR values are unique
- Build the array using:

$$a_i = px_i \oplus px_{i-1}$$

This guarantees:

- XOR of subarray (l, r) is 0
- No other subarray XOR becomes 0

—

9.9 Suffix Sums and XOR

9.9.1 Definition

Suffix Sum is defined as:

$$s_i = a_i + a_{i+1} + \cdots + a_n$$

Suffix XOR is defined as:

$$sx_i = a_i \oplus a_{i+1} \oplus \cdots \oplus a_n$$

Suffix sums and suffix XOR work in the same way as prefix sums and prefix XOR, but are computed from the end of the array instead of the beginning.

9.9.2 Relation with Prefix Arrays

If p_i is the prefix sum and px_i is the prefix XOR, then:

$$s_i = p_n - p_{i-1}$$

$$sx_i = px_n \oplus px_{i-1}$$

—

10 Pigeonhole Principle

10.1 Statement

10.1.1 Basic Form

If more than n objects are placed into n boxes, then at least one box contains more than one object.

Formally:

If $k > n$ objects are distributed into n boxes, $\Rightarrow \exists$ a box with at least 2 objects.

—

10.1.2 Generalized Form

If k objects are placed into n boxes, then at least one box contains at least:

$$\left\lceil \frac{k}{n} \right\rceil$$

objects.

—

10.2 Why This Works (Intuition)

Assume the opposite:

- Every box contains at most $\left\lfloor \frac{k}{n} \right\rfloor$ objects
- Then total objects $\leq n \cdot \left\lfloor \frac{k}{n} \right\rfloor < k$

This contradicts the assumption that k objects exist.
Hence, the principle must hold.

—

10.3 Simple Examples

10.3.1 Example 1

Among 13 people, at least two people have birthdays in the same month.

Explanation

- Objects: 13 people
- Boxes: 12 months
- Since $13 > 12$, some month has at least 2 birthdays

—

10.3.2 Example 2

If 11 integers are chosen from $\{1, 2, \ldots, 10\}$, at least one number is repeated.

—

10.4 Classic CP Use Case

Subarray Sum Divisible by n Given an array of length n, consider prefix sums modulo n:

$$p_i = (a_1 + a_2 + \cdots + a_i) \bmod n$$

There are $n + 1$ values:

$$p_0, p_1, \ldots, p_n$$

But only n possible remainders.
By the Pigeonhole Principle:

- Either some $p_i = 0$
- Or two prefix sums are equal

In both cases, a subarray with sum divisible by n exists.

—

10.5 Practice Problem (Pigeonhole Principle)

Problem CF 2120D — Matrix Game

Link `https://codeforces.com/contest/2120/problem/D`

Main Idea Each cell of the matrix can take one of k values. Harshith tries to avoid creating an $a \times b$ submatrix with all equal elements.

- Consider all possible placements of values in the matrix
- The number of submatrices of size $a \times b$ grows with n and m
- The number of distinct value patterns Harshith can enforce is limited by k

When the matrix becomes large enough, the number of required configurations exceeds the number of distinct value choices.

By the **Pigeonhole Principle**, some $a \times b$ submatrix must have all equal elements, guaranteeing Aryan's win.

Approach

- Observe that Harshith controls the matrix values, but each cell can take only one of k distinct values
- Aryan wins if there exists an $a \times b$ submatrix with all equal values
- Count the number of distinct $a \times b$ submatrices in an $n \times m$ matrix
- Compare this with the maximum number of distinct value assignments Harshith can use to avoid repetition

Once the number of required submatrices exceeds the number of distinct patterns possible using k values, avoiding repetition becomes impossible.

By the **Pigeonhole Principle**, at least one $a \times b$ submatrix must have all elements equal, and Aryan is guaranteed to win.

Key Insight This is an extremal construction problem:

- If objects (submatrices) > boxes (distinct value patterns)
- Then repetition is unavoidable

The task is to find the smallest (n, m) where this inevitability occurs.

11 Basic Geometry for Competitive Programming

11.1 Distance Between Two Points

Let two points be:

$$A(x_1, y_1), \quad B(x_2, y_2)$$

—

11.1.1 Euclidean Distance

Formula

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Important Trick Instead of comparing distances, compare **squared distances**:

$$(x_1 - x_2)^2 + (y_1 - y_2)^2$$

to avoid floating-point inaccuracies.

—

11.1.2 Manhattan Distance

Formula

$$d = |x_1 - x_2| + |y_1 - y_2|$$

Interpretation Distance when movement is restricted to horizontal and vertical steps.

11.2 Practice Problem (Geometry)

Problem **CF 2119B — Line Segments**

Link `https://codeforces.com/contest/2119/problem/B`

Main Idea Each move allows you to travel *exactly* a fixed distance, but in any direction. Thus, after all operations, the question reduces to whether the final point can be reached using segments of given lengths.

Let:

- D be the Euclidean distance between the start and end points
- $S = a_1 + a_2 + \cdots + a_n$ be the total available movement
- $M = \max(a_1, a_2, \ldots, a_n)$ be the largest single move

Approach Using the triangle inequality:

- The farthest reachable distance is S
- The closest reachable distance is $\max(0,\, 2M - S)$

Reaching the target point is possible if and only if:

$$\max(0,\, 2M - S) \leq D \leq S$$

Why It Works Each move can be rotated arbitrarily, so only the total length and imbalance between segments matters. This is a direct application of geometric feasibility using triangle inequality constraints. —

12 Basic Statistics

12.1 Mean

12.1.1 Definition

$$\text{Mean} = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

12.1.2 Uses

- Expected value problems
- Average operations
- Probability calculations

—

12.2 Median

12.2.1 Definition

The median is the middle element after sorting the array.

- If n is odd: element at position $\frac{n}{2}$
- If n is even: any value between the two middle elements

—

12.2.2 Key Property

The median minimizes:

$$\sum |x_i - m|$$

This makes it extremely important in optimization problems. —

12.3 Practice Problem (Median)

Problem **CF 2149D — A and B**

Link `https://codeforces.com/contest/2149/problem/D`

Main Idea We want all characters of one type (either all `a` or all `b`) to form a single contiguous block using the minimum number of adjacent swaps.

This is equivalent to:

- Fixing a target block of consecutive positions
- Moving all chosen characters into this block
- Minimizing the total number of swaps (movement distance)

Key Observation Adjacent swaps correspond to **absolute movement cost**. Thus, minimizing swaps is the same as minimizing:

$$\sum |pos_i - target_i|$$

This sum is minimized when the block is centered at the **median** of the chosen character positions.

Approach

- Collect indices of all `a`'s (or all `b`'s)
- Assume they form a contiguous block
- Align them to consecutive positions
- The optimal alignment is achieved by placing the block around the median
- Compute total movement cost
- Do this for both characters and take the minimum

Why It Works The median minimizes the sum of absolute differences. Since each adjacent swap shifts a character by one position, the total number of swaps is minimized by median alignment.

12.4 Practice Problem (Median & Manhattan Distance)

Problem CF 2122C — Manhattan Pairs

Link `https://codeforces.com/problemset/problem/2122/C`

Main Idea We are given n points on a 2D plane (n is even), and must form $\frac{n}{2}$ disjoint pairs to **maximize the total Manhattan distance**:

$$\sum |x_a - x_b| + |y_a - y_b|$$

The key observation is that Manhattan distance is separable across dimensions, and optimal pairing is achieved by pairing **far-apart points**.

Key Observations

- Sorting points by x separates them spatially
- Pairing smallest x with largest x maximizes $|x_a - x_b|$
- After fixing x separation, pairing extremes in y further maximizes $|y_a - y_b|$
- This mirrors the idea that distance is maximized when elements are placed on opposite sides of the median

Greedy Strategy

- Sort all points by x
- Split into two halves:
  - Left half = smaller x
  - Right half = larger x
- Sort both halves by y
- Pair:
$$\text{left}[i] \text{ with } \text{right}[\tfrac{n}{2} - 1 - i]$$
- This pairs smallest y with largest y, maximizing vertical distance

Why This Works For Manhattan distance:

$$|x_1 - x_2| + |y_1 - y_2|$$

both components are maximized independently by pairing elements symmetrically around their medians.

—

12.5 Mode

12.5.1 Definition

The mode is the most frequent element in a dataset.

12.5.2 Uses

- Frequency-based decisions
- Majority element problems
- Hash map counting problems

12.6 Practice Problem: Majority Element

Problem **LeetCode — Majority Element**

Link `https://leetcode.com/problems/majority-element/description/`

Core Idea A "majority element" is an element that appears **more than $\lfloor n/2 \rfloor$ times** in an array of length n.

Key observations:

- The majority element is unique if it exists.
- A naive frequency map takes $O(n)$ time and $O(n)$ space.
- There exists an $O(n)$ time and $O(1)$ space algorithm: the **Boyer–Moore Voting Algorithm**.

Boyer–Moore works by maintaining a candidate and a counter:

$$\text{if counter } = 0, \text{ set candidate} = \text{current element}$$

$$\text{if current} == \text{candidate, counter++ else counter–}$$

The final candidate is the majority element if a majority element exists (since it survives net cancellation).

12.7 Practice Problem: Majority Element II

Problem **LeetCode — Majority Element II**

Link `https://leetcode.com/problems/majority-element-ii/description/`

Core Idea This is a generalization: find all elements that appear **more than $\lfloor n/3 \rfloor$ times**.

There can be at most two such elements. The optimized $O(n)$, $O(1)$ space method extends Boyer–Moore:

- Maintain two candidates and two counters.
- First pass: adjust candidates/counters by cancellation.
- Second pass: verify actual counts.

—

13 Additional Practice Problems

Suffix Sums & Combinatorics : CF 2008F
https://codeforces.com/contest/2008/problem/F

Bitwise Operations : CF 1614C
https://codeforces.com/problemset/problem/1614/C

Factors : CF 1925B
https://codeforces.com/contest/1925/problem/B

Combinatorics : CF 2150B
https://codeforces.com/problemset/problem/2150/B

Brute Force : CF 2144D
https://codeforces.com/problemset/problem/2144/D

Maths (Log2) : CF 2013B
https://codeforces.com/problemset/problem/2109/B

Binary Numbers : CF 1225C
https://codeforces.com/problemset/problem/1225/C

Factors : CF 2193E
https://codeforces.com/problemset/problem/2193/E

Bitwise : CF 2189C2
https://codeforces.com/problemset/problem/2189/C2

Telescopic Series : CF 2183E
https://codeforces.com/problemset/problem/2183/E

GCD : CF 2116C
https://codeforces.com/contest/2116/problem/C

Digits : CF 2132D
https://codeforces.com/problemset/problem/2132/D

Prefix/ Suffix GCD : CF 2126E
https://codeforces.com/problemset/problem/2126/E

Arithemetic Progression : CF 2117D
https://codeforces.com/problemset/problem/2117/D

Number Theory CF 2109C2
https://codeforces.com/problemset/problem/2109/C2